

b2xrot

June 6, 2026

compile_options	-lang cpp -fpga-mem-th 4 -ct 1 -es 1 -mcd 16 -mdd 1024 -mdy 33 -single -
filename	b2xrot.dsp
maths.lib/author	GRAME
maths.lib/copyright	GRAME
maths.lib/license	LGPL with exception
maths.lib/name	Faust Math Library
maths.lib/version	2.9.0
name	b2xrot
seam.ambisonics.lib/author	Giuseppe Silvi
seam.ambisonics.lib/license	CC3
seam.ambisonics.lib/name	SEAM Ambisonics Library
seam.ambisonics.lib/version	0.1
seam.lib/author	Giuseppe Silvi, Davide Tedesco
seam.lib/license	CC3
seam.lib/name	Faust SEAM main lib
seam.lib/version	0.2

This document provides a mathematical description of the Faust program text stored in the `b2xrot.dsp` file. See the notice in Section 3 (page 2) for details.

1 Mathematical definition of process

The *b2xrot* program evaluates the signal transformer denoted by **process**, which is mathematically defined as follows:

1. Output signals y_i for $i \in [1, 4]$ such that

$$y_1(t) = 1.4142135623731 \cdot x_1(t)$$

$$y_2(t) = p_2(t) \cdot s_1(t) - p_6(t) \cdot s_3(t)$$

$$y_3(t) = p_8(t) \cdot s_2(t) + p_9(t) \cdot x_2(t)$$

$$y_4(t) = p_6(t) \cdot s_1(t) + p_2(t) \cdot s_3(t)$$

2. Input signals x_i for $i \in [1, 4]$
3. User-interface input signals u_{s_i} for $i \in [1, 3]$ such that

$$\begin{array}{ll}
\text{"YAW Rotation around Z-axis"} & u_{s_1}(t) \in [-180, 180] \quad (\text{default value} = 0) \\
\text{"ROLL Rotation around X-axis"} & u_{s_2}(t) \in [-180, 180] \quad (\text{default value} = 0) \\
\text{"PITCH Rotation around Y-axis"} & u_{s_3}(t) \in [-180, 180] \quad (\text{default value} = 0)
\end{array}$$

4. Intermediate signals p_i for $i \in [1, 9]$ and s_i for $i \in [1, 3]$ such that

$$\begin{aligned}
p_1(t) &= 0.0174532925199433 \cdot u_{s_1}(t) \\
p_2(t) &= \cos(p_1(t)) \\
p_3(t) &= 0.0174532925199433 \cdot u_{s_2}(t) \\
p_4(t) &= \cos(p_3(t)) \\
p_5(t) &= \sin(p_3(t)) \\
p_6(t) &= \sin(p_1(t)) \\
p_7(t) &= 0.0174532925199433 \cdot u_{s_3}(t) \\
p_8(t) &= \cos(p_7(t)) \\
p_9(t) &= \sin(p_7(t))
\end{aligned}$$

$$\begin{aligned}
s_1(t) &= (p_4(t) \cdot x_3(t) - p_5(t) \cdot x_4(t)) \\
s_2(t) &= (p_5(t) \cdot x_3(t) + p_4(t) \cdot x_4(t)) \\
s_3(t) &= (p_8(t) \cdot x_2(t) - p_9(t) \cdot s_2(t))
\end{aligned}$$

2 Block diagram of process

The block diagram of `process` is shown on Figure 1 (page 3).

3 Notice

- This document was generated using Faust version 2.85.5 on June 06, 2026.
- The value of a Faust program is the result of applying the signal transformer denoted by the expression to which the `process` identifier is bound to input signals, running at the f_s sampling frequency.

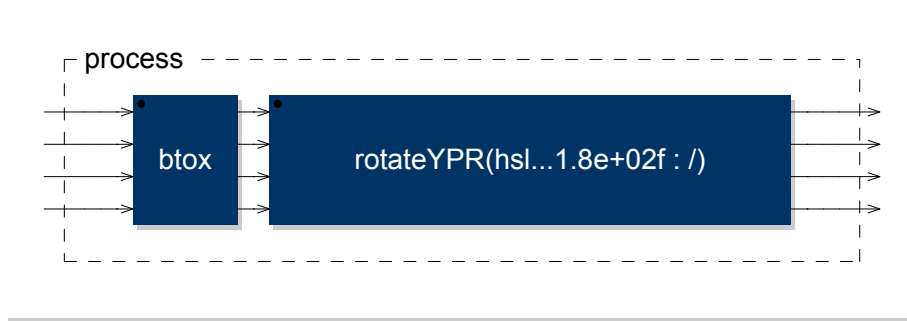


Figure 1: Block diagram of `process`

- Faust (*Functional Audio Stream*) is a functional programming language designed for synchronous real-time signal processing and synthesis applications. A Faust program is a set of bindings of identifiers to expressions that denote signal transformers. A signal s in S is a function mapping¹ times $t \in \mathbb{Z}$ to values $s(t) \in \mathbb{R}$, while a signal transformer is a function from S^n to S^m , where $n, m \in \mathbb{N}$. See the Faust manual for additional information (<http://faust.grame.fr>).
- Every mathematical formula derived from a Faust expression is assumed, in this document, to having been normalized (in an implementation-dependent manner) by the Faust compiler.
- A block diagram is a graphical representation of the Faust binding of an identifier I to an expression E ; each graph is put in a box labeled by I . Subexpressions of E are recursively displayed as long as the whole picture fits in one page.
- The `b2xrot-mdoc/` directory may also include the following subdirectories:
 - `cpp/` for Faust compiled code;
 - `pdf/` which contains this document;
 - `src/` for all Faust sources used (even libraries);
 - `svg/` for block diagrams, encoded using the Scalable Vector Graphics format (<http://www.w3.org/Graphics/SVG/>);
 - `tex/` for the $\text{\LaTeX}$ source of this document.

4 Faust code listings

This section provides the listings of the Faust code used to generate this document, including dependencies.

¹Faust assumes that $\forall s \in S, \forall t \in \mathbb{Z}, s(t) = 0$ when $t < 0$.

Listing 1: b2xrot.dsp

```

1 import("seam.lib");
2 // Canonical DSP: sam.btox + sam.rotateYPR in seam.ambisonics.lib
3 yaw = hslider("[00]YAW Rotation around Z-axis",0,-180,180,0.5) : ma.deg2rad;
4 pitch = hslider("[01]PITCH Rotation around Y-axis",0,-180,180,0.5) : ma.deg2rad;
5 roll = hslider("[02]ROLL Rotation around X-axis",0,-180,180,0.5) : ma.deg2rad;
6 process = sam.btox : sam.rotateYPR(yaw, pitch, roll);

```

Listing 2: seam.lib

```

1 declare name "Faust SEAM main lib";
2 declare version "0.2";
3 declare author "Giuseppe Silvi";
4 declare author "Davide Tedesco";
5 declare license "CC3";
6 // calling standard faust libraries
7 import("stdfaust.lib");
8 // ### standard faust lib extensions
9 san = library("seam.analyzers.lib");
10 sba = library("seam.basic.lib");
11 sma = library("seam.math.lib");
12 sfi = library("seam.filters.lib");
13 sre = library("seam.reverbs.lib");
14 sno = library("seam.noises.lib");
15 // ### general Ambisonics theory
16 sam = library("seam.ambisonics.lib");
17 sdw = library("seam.dwt.lib");
18 // ### stereophony and perception
19 sst = library("seam.stereophony.lib");
20 // shr = library("seam.hrtf.lib");
21 // ### CSound and MaxMsp object cloning
22 scs = library("seam.csound.lib");
23 scy = library("seam.cyclone.lib");
24 // ### author specific literature
25 // sds = library("seam.discipio.lib");
26 smg = library("seam.gerzon.lib");
27 // sln = library("seam.nono.lib");
28 // sgn = library("seam.nottoli.lib");
29 sjm = library("seam.moorer.lib");
30 scr = library("seam.roads.lib");
31 sms = library("seam.schroeder.lib");
32 sfv = library("seam.freeverb.lib");
33
34 sff = library("seam.ffunctions.lib");
35
36 // ### live electronics stuff
37 // sgu = library("seam.gui.lib");
38 // shw = library("seam.hardware.lib");
39 // smx = library("seam.mixer.lib");
40
41 // ### instrument specific literature
42 // svc = library("seam.vcs3.lib");

```

Listing 3: stdfaust.lib

```

1 //##### stdfaust.lib #####
2 // The purpose of this library is to give access to all the Faust standard libraries
3 // through a series of environments.
4 //#####

```

```

5
6 aa = library("aanl.lib");
7 sf = library("all.lib");
8 an = library("analyzers.lib");
9 ba = library("basics.lib");
10 co = library("compressors.lib");
11 db = library("debug.lib");
12 de = library("delays.lib");
13 dm = library("demos.lib");
14 dx = library("dx7.lib");
15 en = library("envelopes.lib");
16 fd = library("fds.lib");
17 fi = library("filters.lib");
18 ho = library("hoa.lib");
19 hy = library("hysteresis.lib");
20 it = library("interpolators.lib");
21 la = library("linearalgebra.lib");
22 ma = library("maths.lib");
23 mi = library("mi.lib");
24 mo = library("motion.lib");
25 ef = library("misceffects.lib");
26 no = library("noises.lib");
27 os = library("oscillators.lib");
28 pf = library("phaflangers.lib");
29 pl = library("platform.lib");
30 pm = library("physmodels.lib");
31 qu = library("quantizers.lib");
32 rm = library("reducemaps.lib");
33 re = library("reverbs.lib");
34 ro = library("routes.lib");
35 sp = library("spats.lib");
36 si = library("signals.lib");
37 so = library("soundfiles.lib");
38 sy = library("synths.lib");
39 ve = library("vaeffects.lib");
40 vl = library("version.lib");
41 wa = library("webaudio.lib");
42 wd = library("wdmodels.lib");

```

Listing 4: seam.ambisonics.lib

```

1 declare name "SEAM Ambisonics Library";
2 declare version "0.1";
3 declare author "Giuseppe Silvi";
4 declare license "CC3";
5 //
6 // SEAM Ambisonics Library
7 //
8 // Functions for first-order Ambisonics processing in ACN/SN3D (AmbiX)
9 // format, including format conversion from B-format (FuMa) and
10 // 3D rotation.
11 //
12 // Channel ordering follows the ACN convention:
13 // a0 = W (omnidirectional)
14 // a1 = Y (front-back, left-right)
15 // a2 = Z (up-down)
16 // a3 = X (front-back)
17 //
18 // References:
19 // - Furuse-Malham, "A 3-D Sound Primer" (1998)
20 // - Nachbar et al., "ambiX -- A Suggested Ambisonics Format" (2011)
21 // - Gerzon, "Ambisonics. Part two: Studio techniques" (1975)
22 // - Malham, "3-D Sound Spatialization using Ambisonic Techniques" (1999)
23 //

```

```

24 //
25 //
26 import("seam.lib");
27 //
28 sam = library("seam.ambisonics.lib");
29 //
30 //           B-format to AmbiX conversion
31 //
32 // Converts a first-order Ambisonics signal from B-format (FuMa)
33 // to AmbiX (ACN/SN3D).
34 //
35 // B-format (FuMa): W, X, Y, Z -- with W attenuated by 1/sqrt(2)
36 // AmbiX (ACN/SN3D): W, Y, Z, X -- with W at unity gain
37 //
38 // The conversion performs two operations:
39 // 1. Rescale W by sqrt(2) to restore unity gain
40 // 2. Reorder channels from FuMa (WXYZ) to ACN (WYZX)
41 //
42 btow(w,x,y,z) = a0, a1, a2, a3
43 with {
44     a0 = w * sqrt(2); // W: restore unity gain
45     a1 = y;           // Y: FuMa -> ACN reorder
46     a2 = z;           // Z: unchanged
47     a3 = x;           // X: FuMa -> ACN reorder
48 };
49 //
50 //           AmbiX -> tetrahedral decoder
51 //
52 // Decodes a first-order AmbiX signal to four loudspeakers placed at
53 // the vertices of a tetrahedron inscribed in a cube:
54 //
55 // LFU -- Left Front Up   ( +Y +Z +X )
56 // RFD -- Right Front Down ( -Y -Z +X )
57 // RBU -- Right Back Up   ( -Y +Z -X )
58 // LBD -- Left Back Down  ( +Y -Z -X )
59 //
60 // Each output is the sum of W and the three Cartesian components
61 // signed according to the corner's position, scaled by 1/2:
62 //
63 // LFU = (a0 + a1 + a2 + a3) / 2
64 // RFD = (a0 - a1 - a2 + a3) / 2
65 // RBU = (a0 - a1 + a2 - a3) / 2
66 // LBD = (a0 + a1 - a2 - a3) / 2
67 //
68 // The matrix is the AmbiX-domain analogue of Gerzon's `bamodule`
69 // (FuMa B-format -> tetrahedron) found in seam.gerzon.lib. The two
70 // differ only in the input reference system; the same diagonal-cube
71 // vertex layout is preserved.
72 //
73 // Compensation shelving filters (Gerzon 1975) are deliberately
74 // omitted: the STONE tetrahedral loudspeaker amplifier handles
75 // frequency-domain correction downstream.
76 //
77 // References:
78 // - Gerzon, "Ambisonics. Part two: Studio techniques" (1975)
79 // - Malham, "Space in Music -- Music in Space" (1998)
80 //
81 bamodulex(a0,a1,a2,a3) = lfu, rfd, rbu, lbd
82 with {
83     lfu = (a0 + a1 + a2 + a3) / 2;
84     rfd = (a0 - a1 - a2 + a3) / 2;
85     rbu = (a0 - a1 + a2 - a3) / 2;
86     lbd = (a0 + a1 - a2 - a3) / 2;
87 };
88 //
89 //           First-order AmbiX rotation
90 //
91 // Applies 3D rotation to a first-order AmbiX signal.

```

```

92 // Rotation is applied in intrinsic order: Roll -> Pitch -> Yaw.
93 // The resulting matrix is R = Rz(yaw) * Ry(pitch) * Rx(roll).
94 //
95 // Roll (X-axis) -- lateral tilt (left-right vertical plane)
96 // Pitch (Y-axis) -- forward-backward tilt (vertical plane)
97 // Yaw (Z-axis) -- horizontal rotation (azimuth)
98 //
99 // Channel W (omnidirectional) is unaffected by rotation.
100 //
101 // Arguments:
102 // yaw, pitch, roll -- rotation angles in radians
103 // a0, a1, a2, a3 -- input channels in ACN order (W, Y, Z, X)
104 //
105 rotateYPR(yaw, pitch, roll, a0, a1, a2, a3) =
106     a0, a1_ypr, a2_ypr, a3_ypr
107 with {
108     // Roll (X-axis): rotates Y and Z
109     a1_r = a1*cos(roll) - a2*sin(roll);
110     a2_r = a1*sin(roll) + a2*cos(roll);
111     a3_r = a3;
112     // Pitch (Y-axis): rotates Z and X
113     a1_rp = a1_r;
114     a2_rp = a2_r*cos(pitch) + a3_r*sin(pitch);
115     a3_rp = -a2_r*sin(pitch) + a3_r*cos(pitch);
116     // Yaw (Z-axis): rotates Y and X
117     a1_ypr = a1_rp*cos(yaw) - a3_rp*sin(yaw);
118     a2_ypr = a2_rp;
119     a3_ypr = a1_rp*sin(yaw) + a3_rp*cos(yaw);
120 };
121 //
122 //----- UHJ C-FORMAT -----
123 // AmbiX (ACN/SN3D) -> UHJ C-format L,R,T,Q.
124 // Gerzon, "Ambisonics in Multichannel Broadcasting and Video", JAES 1983.
125 // Quadrature all-pass pair re-derived analytically (SEAM-LTM x2uhj, 2026);
126 // max phase error 1.36 deg over 20 Hz-20 kHz. Coefficients are the single
127 // source of truth in seam-ltm plugins/x2uhj/source/x2uhj_coeffs.h.
128 // NB canonical structure: every component passes the real network hR; only the
129 // j-terms use the imaginary network hI. The 90 deg is the DIFFERENCE between
130 // the two matched networks. Earlier drafts leaving the real path unfiltered
131 // are superseded.
132 //
133 // 2nd-order RBJ all-pass parameterised by (frequency, Q):
134 apuhj(f,Q) = fi.tf2(a2, a1, 1.0, a1, a2)
135 with {
136     w = 2.0*ma.PI*f/ma.SR;
137     alpha = sin(w)/(2.0*Q);
138     n = 1.0 + alpha;
139     a1 = -2.0*cos(w)/n;
140     a2 = (1.0 - alpha)/n;
141 };
142 // Matched quadrature pair (3 sections each):
143 hRuhj = apuhj(141.887012,0.201947) : apuhj(671.745773,0.212154) : apuhj
144     (18654.238494,0.303097);
145 hIuhj = apuhj(24.003923,0.309038) : apuhj(2991.937233,0.384766) : apuhj
146     (3219.986192,0.096342);
147 // AmbiX ACN inputs in order (W,Y,Z,X) -> UHJ (L,R,T,Q):
148 x2uhj(aW,aY,aZ,aX) = L,R,T,Q
149 with {
150     W = aW/sqrt(2.0); X = aX; Y = aY; Z = aZ; // ACN/SN3D -> FuMa
151     Wr = hRuhj(W); Xr = hRuhj(X); Yr = hRuhj(Y); Zr = hRuhj(Z);
152     Wi = hIuhj(W); Xi = hIuhj(X);
153     S = 0.9396926*Wr + 0.1855740*Xr;
154     D = (-0.3420201*Wi + 0.5098604*Xi) + 0.6554516*Yr;
155     L = 0.5*(S + D);
156     R = 0.5*(S - D);
157     T = (-0.1432*Wi + 0.6512*Xi) - 0.7071*Yr;
158     Q = 0.9772*Zr;
159 };

```

```

158 //process = x2uhj;
159 //
160 //----- MONO -> AMBIX (HAAR) ---
161 // Mono to first-order AmbiX via the Haar wavelet decomposition
162 // (sdw.haarmn from seam.dwt.lib). Channel 0 (W) is the passthrough; the
163 // QMF sub-bands form the directional components. Rotation, if any, is
164 // applied separately with rotateYPR. SEAM-LTM M2XHGR plugin.
165 m2xhgr = sdw.haarmn(1);
166 //process = no.noise : m2xhgr;
167 //
168 //----- STEREO -> AMBIX (HAAR) ---
169 // Stereo (L,R) to first-order AmbiX via Haar, with a divergence control.
170 // Each channel is Haar-encoded (m2xhgr), rotated by +/-divergence, summed,
171 // then globally rotated by yaw. divergence/yaw/pitch/roll in radians.
172 // SEAM-LTM LR2XHGR plugin.
173 lr2xhgr(divergence, yaw, pitch, roll) =
174   par(i, 2, m2xhgr) :
175     rotateYPR(divergence, pitch, roll),
176     rotateYPR(0 - divergence, pitch, roll)
177   :> si.bus(4) :
178     rotateYPR(yaw, 0, 0);
179 //process = (no.noise, no.noise) : lr2xhgr(ma.PI/2, 0, 0, 0);

```

Listing 5: maths.lib

```

1 //##### maths.lib #####
2 // Maths library. Its official prefix is `ma`.
3 //
4 // This library provides mathematical functions and utilities for numerical
5 // computations in Faust. It includes trigonometric, exponential, logarithmic, and
6 // statistical functions, constants, and complex-number
7 // operations used throughout Faust DSP and control code.
8 //
9 // The Maths library is organized into 1 section:
10 //
11 // * [Functions Reference](#functions-reference)
12 //
13 // Some functions are implemented as Faust foreign functions of `math.h` functions
14 // that are not part of Faust's primitives. Defines also various constants and several
15 // utilities.
16 //
17 // #### References
18 //
19 // * <https://github.com/grame-cncm/faustlibraries/blob/master/maths.lib>
20 //#####
21
22 // ## History
23 // * 06/13/2016 [RM] normalizing and integrating to new libraries
24 // * 07/08/2015 [Y0] documentation comments
25 // * 20/06/2014 [SL] added FTZ function
26 // * 22/06/2013 [Y0] added float|double|quad variants of some foreign functions
27 // * 28/06/2005 [Y0] postfix functions with 'f' to force float version instead of double
28 // * 28/06/2005 [Y0] removed 'modf' because it requires a pointer as argument
29
30 /*****
31 *****/
32 FAUST library file
33 Copyright (C) 2003-2016 GRAME, Centre National de Creation Musicale
34 -----
35 This program is free software; you can redistribute it and/or modify
36 it under the terms of the GNU Lesser General Public License as
37 published by the Free Software Foundation; either version 2.1 of the
38 License, or (at your option) any later version.
39

```



```

40 This program is distributed in the hope that it will be useful,
41 but WITHOUT ANY WARRANTY; without even the implied warranty of
42 MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
43 GNU Lesser General Public License for more details.
44
45 You should have received a copy of the GNU Lesser General Public
46 License along with the GNU C Library; if not, write to the Free
47 Software Foundation, Inc., 59 Temple Place, Suite 330, Boston, MA
48 02111-1307 USA.
49
50 EXCEPTION TO THE LGPL LICENSE : As a special exception, you may create a
51 larger FAUST program which directly or indirectly imports this library
52 file and still distribute the compiled code generated by the FAUST
53 compiler, or a modified version of this compiled code, under your own
54 copyright and license. This EXCEPTION TO THE LGPL LICENSE explicitly
55 grants you the right to freely choose the license for the resulting
56 compiled code. In particular the resulting compiled code has no obligation
57 to be LGPL or GPL. For example you are free to choose a commercial or
58 closed source license or any other license if you decide so.
59 *****
60 *****/
61
62 // This library contains platform specific constants
63 ba = library("basics.lib");
64 pl = library("platform.lib");
65 ma = library("maths.lib"); // for compatible copy/paste out of this file
66
67 declare name "Faust Math Library";
68 declare version "2.9.0";
69 declare author "GRAMME";
70 declare copyright "GRAMME";
71 declare license "LGPL with exception";
72
73 //=====Functions Reference=====
74 //=====
75
76 //-----`(ma.)SR`-----
77 // Current sampling rate given at init time. Constant during program execution.
78 //
79 // #### Usage
80 //
81 // ```
82 // SR : _
83 // ```
84 //
85 // Where:
86 //
87 // * `SR`: initialization-time sampling rate constant
88 //
89 // #### Test
90 // ```
91 // ma = library("maths.lib");
92 // SR_test = ma.SR;
93 // ```
94 //-----
95 SR = pl.SR;
96
97 //-----`(ma.)T`-----
98 // Current sample duration in seconds computed from the sampling rate given at init time.
99 // Constant during program execution.
100 //
101 // #### Usage
102 //
103 // ```
104 // T : _
105 // ```
106 //

```

```

107 // Where:
108 //
109 // * `T`: sample duration (`1/SR`) constant
110 //
111 // #### Test
112 // ```
113 // ma = library("maths.lib");
114 // T_test = ma.T;
115 // ```
116 //-----
117 T = 1.0 / SR;
118
119 //-----`(ma.)BS`-----
120 // Current block-size. Can change during the execution at each block.
121 //
122 // #### Usage
123 //
124 // ```
125 // BS : _
126 // ```
127 //
128 // Where:
129 //
130 // * `BS`: current processing block size
131 //
132 // #### Test
133 // ```
134 // ma = library("maths.lib");
135 // BS_test = ma.BS;
136 // ```
137 //-----
138 BS = pl.BS;
139
140 //-----`(ma.)PI`-----
141 // Constant PI in double precision.
142 //
143 // #### Usage
144 //
145 // ```
146 // PI : _
147 // ```
148 //
149 // Where:
150 //
151 // * `PI`: double-precision constant
152 //
153 // #### Test
154 // ```
155 // ma = library("maths.lib");
156 // PI_test = ma.PI;
157 // ```
158 //-----
159 PI = 3.14159265358979323846;
160
161 //-----`(ma.)deg2rad`-----
162 // Convert degrees to radians.
163 //
164 // #### Usage
165 //
166 // ```
167 // 45. : deg2rad
168 // ```
169 //
170 // Where:
171 //
172 // * input: angle in degrees to convert

```

```

175 //
176 // #### Test
177 // ```
178 // ma = library("maths.lib");
179 // deg2rad_test = 45.0 : ma.deg2rad;
180 // ```
181 //-----
182 deg2rad = _ * PI / 180.;
183
184
185 //-----`(ma.)rad2deg`-----
186 // Convert radians to degrees.
187 //
188 // #### Usage
189 //
190 // ```
191 // ma.PI : rad2deg
192 // ```
193 //
194 // Where:
195 //
196 // * input: angle in radians to convert
197 //
198 // #### Test
199 // ```
200 // ma = library("maths.lib");
201 // rad2deg_test = ma.PI : ma.rad2deg;
202 // ```
203 //-----
204 rad2deg = _ * 180. / PI;
205
206
207 //-----`(ma.)E`-----
208 // Constant e in double precision.
209 //
210 // #### Usage
211 //
212 // ```
213 // E : _
214 // ```
215 //
216 // Where:
217 //
218 // * `E`: double-precision Euler's number constant
219 //
220 // #### Test
221 // ```
222 // ma = library("maths.lib");
223 // E_test = ma.E;
224 // ```
225 //-----
226 E = 2.71828182845904523536;
227
228
229 //-----`(ma.)EPSILON`-----
230 // Constant EPSILON available in simple/double/quad precision,
231 // as defined in the [floating-point standard](https://en.wikipedia.org/wiki/IEEE_754)
232 // and [machine epsilon](https://en.wikipedia.org/wiki/Machine_epsilon),
233 // that is smallest positive number such that `1.0 + EPSILON != 1.0`.
234 //
235 // #### Usage
236 //
237 // ```
238 // EPSILON : _
239 // ```
240 //
241 // Where:
242 //

```

```

243 // * `EPSILON`: machine epsilon constant for the current floating-point precision
244 //
245 // #### Test
246 // ```
247 // ma = library("maths.lib");
248 // EPSILON_test = ma.EPSILON;
249 // ```
250 //-----
251 singleprecision EPSILON = 1.192092896e-07;
252 doubleprecision EPSILON = 2.2204460492503131e-016;
253 quadprecision EPSILON = 1.084202172485504434007452e-019;
254 fixedpointprecision EPSILON = 2.2204460492503131e-016;
255
256 //-----`(ma.)MIN`-----
257 // Constant MIN available in simple/double/quad precision (minimal positive value).
258 //
259 // #### Usage
260 //
261 // ```
262 // MIN : _
263 // ```
264 //
265 // Where:
266 //
267 // * `MIN`: minimal positive normalized value for the current precision
268 //
269 // #### Test
270 // ```
271 // ma = library("maths.lib");
272 // MIN_test = ma.MIN * 1e307;
273 // ```
274 //-----
275 singleprecision MIN = 1.175494351e-38;
276 doubleprecision MIN = 2.2250738585072014e-308;
277 quadprecision MIN = 2.2250738585072014e-308;
278 fixedpointprecision MIN = 2.2250738585072014e-308;
279
280 //-----`(ma.)MAX`-----
281 // Constant MAX available in simple/double/quad precision (maximal positive value).
282 //
283 // #### Usage
284 //
285 // ```
286 // MAX : _
287 // ```
288 //
289 // Where:
290 //
291 // * `MAX`: maximal finite value for the current precision
292 //
293 // #### Test
294 // ```
295 // ma = library("maths.lib");
296 // MAX_test = ma.MAX;
297 // ```
298 //-----
299 singleprecision MAX = 3.402823466e+38;
300 doubleprecision MAX = 1.7976931348623158e+308;
301 quadprecision MAX = 1.7976931348623158e+308;
302 fixedpointprecision MAX = 1.7976931348623158e+308;
303
304 // Obsolete, kept for compatibility reasons
305 INFINITY = MAX;
306
307 //-----`(ma.)FTZ`-----
308 // Flush to zero: force samples under the "maximum subnormal number"
309
310

```

```

311 // to be zero. Usually not needed in C++ because the architecture
312 // file take care of this, but can be useful in JavaScript for instance.
313 //
314 // #### Usage
315 //
316 // ```
317 // _ : FTZ : _
318 // ```
319 //
320 // Where:
321 //
322 // * `x`: input signal to flush if its magnitude is subnormal
323 //
324 // #### Test
325 // ```
326 // ma = library("maths.lib");
327 // FTZ_test = ((ma.MIN * 0.5) : ma.FTZ), ((ma.MIN * 1e307) : ma.FTZ);
328 // ```
329 //
330 // #### References
331 //
332 // * <http://docs.oracle.com/cd/E19957-01/806-3568/ncg\_math.html>
333 //-----
334 FTZ(x) = x * (abs(x) > MIN);
335
336
337 //-----`(ma.)copysign`-----
338 // Changes the sign of x (first input) to that of y (second input).
339 //
340 // #### Usage
341 //
342 // ```
343 // _ : copysign : _
344 // ```
345 //
346 // Where:
347 //
348 // * `x`: value whose magnitude is preserved
349 // * `y`: value providing the sign
350 //
351 // #### Test
352 // ```
353 // ma = library("maths.lib");
354 // copysign_test = (-1.0, 2.0) : ma.copysign;
355 // ```
356 //-----
357 copysign = ffunction(float copysignf|copysign|copysignl (float, float), <math.h>,"");
358
359
360 //-----`(ma.)neg`-----
361 // Invert the sign (-x) of a signal.
362 //
363 // #### Usage
364 //
365 // ```
366 // _ : neg : _
367 // ```
368 //
369 // Where:
370 //
371 // * `x`: value to negate
372 //
373 // #### Test
374 // ```
375 // ma = library("maths.lib");
376 // neg_test = 3.5 : ma.neg;
377 // ```
378 //-----

```

```

379 neg(x) = -x;
380
381
382 //-----`(ma.)not`-----
383 // Bitwise `not` implemented with [xor](https://faustdoc.grame.fr/manual/syntax/#xor-
    primitive) as `not(x) = x xor -1;`.
384 // So working regardless of the size of the integer, assuming negative numbers in two's
    complement.
385 //
386 // #### Usage
387 //
388 // ```
389 // _ : not : _
390 // ```
391 //
392 // Where:
393 //
394 // * `x`: integer input value
395 //
396 // #### Test
397 //
398 // ma = library("maths.lib");
399 // not_test = 5 : ma.not;
400 // ```
401 //-----
402 not(x) = x xor -1;
403
404
405 //-----`(ma.)sub(x,y)`-----
406 // Subtract `x` and `y`.
407 //
408 // #### Usage
409 //
410 // ```
411 // _ : sub : _
412 // ```
413 //
414 // Where:
415 //
416 // * `x`: first operand
417 // * `y`: second operand
418 //
419 // #### Test
420 //
421 // ma = library("maths.lib");
422 // sub_test = (3, 10) : ma.sub;
423 // ```
424 //-----
425 sub(x,y) = y-x;
426
427
428 //-----`(ma.)inv`-----
429 // Compute the inverse (1/x) of the input signal.
430 //
431 // #### Usage
432 //
433 // ```
434 // _ : inv : _
435 // ```
436 //
437 // Where:
438 //
439 // * `x`: denominator input (non-zero)
440 //
441 // #### Test
442 //
443 // ma = library("maths.lib");
444 // inv_test = 4.0 : ma.inv;

```

```

445 // ``
446 //-----
447 inv(x) = 1/x;
448
449
450 //-----`(ma.)cbrt`-----
451 // Computes the cube root of of the input signal.
452 //
453 // #### Usage
454 //
455 // ``
456 // _ : cbrt : _
457 // ``
458 //
459 // Where:
460 //
461 // * `x`: value whose cube root is computed
462 //
463 // #### Test
464 //
465 // ma = library("maths.lib");
466 // cbrt_test = 8.0 : ma.cbrt;
467 //
468 //-----
469 cbrt = ffunction(float cbrtf|cbrt|cbrtl (float), <math.h>,"");
470
471
472 //-----`(ma.)hypot`-----
473 // Computes the euclidian distance of the two input signals
474 // sqrt(x*x+y*y) without undue overflow or underflow.
475 //
476 // #### Usage
477 //
478 // ``
479 // _ : hypot : _
480 // ``
481 //
482 // Where:
483 //
484 // * `x`: first operand
485 // * `y`: second operand
486 //
487 // #### Test
488 //
489 // ma = library("maths.lib");
490 // hypot_test = (3.0, 4.0) : ma.hypot;
491 //
492 //-----
493 hypot = ffunction(float hypotf|hypot|hypotl (float, float), <math.h>,"");
494
495
496 //-----`(ma.)ldexp`-----
497 // Takes two input signals: x and n, and multiplies x by 2 to the power n.
498 //
499 // #### Usage
500 //
501 // ``
502 // _ : ldexp : _
503 // ``
504 //
505 // Where:
506 //
507 // * `x`: significand input
508 // * `n`: exponent (integer) input
509 //
510 // #### Test
511 //
512 // ma = library("maths.lib");

```

```

513 // ldexp_test = (1.5, 3) : ma.ldexp;
514 // ...
515 //-----
516 ldexp = ffunction(float ldexpf|ldexp|ldexpl (float, int), <math.h>,"");
517
518
519 //-----`(ma.)scalb`-----
520 // Takes two input signals: x and n, and multiplies x by 2 to the power n.
521 //
522 // #### Usage
523 //
524 // ...
525 // _ : scalb : _
526 // ...
527 //
528 // Where:
529 //
530 // * `x`: significand input
531 // * `n`: exponent (integer) input
532 //
533 // #### Test
534 // ...
535 // ma = library("maths.lib");
536 // scalb_test = (2.0, -1) : ma.scalb;
537 // ...
538 //-----
539 scalb = ffunction(float scalbnf|scalbn|scalbnl (float, int), <math.h>,"");
540
541
542 //-----`(ma.)log1p`-----
543 // Computes log(1 + x) without undue loss of accuracy when x is nearly zero.
544 //
545 // #### Usage
546 //
547 // ...
548 // _ : log1p : _
549 // ...
550 //
551 // Where:
552 //
553 // * `x`: offset used in `log(1 + x)` (must be greater than -1)
554 //
555 // #### Test
556 // ...
557 // ma = library("maths.lib");
558 // log1p_test = 0.5 : ma.log1p;
559 // ...
560 //-----
561 log1p = ffunction(float log1pf|log1p|log1pl (float), <math.h>,"");
562
563
564 //-----`(ma.)logb`-----
565 // Return exponent of the input signal as a floating-point number.
566 //
567 // #### Usage
568 //
569 // ...
570 // _ : logb : _
571 // ...
572 //
573 // Where:
574 //
575 // * `x`: positive value whose exponent part is returned
576 //
577 // #### Test
578 // ...
579 // ma = library("maths.lib");
580 // logb_test = 8.0 : ma.logb;

```



```

581 // ``
582 //-----
583 logb = ffunction(float logbf|logb|logbl (float), <math.h>,"");
584
585
586 //-----`(ma.)ilogb`-----
587 // Return exponent of the input signal as an integer number.
588 //
589 // #### Usage
590 //
591 // ``
592 // _ : ilogb : _
593 // ``
594 //
595 // Where:
596 //
597 // * `x`: positive value whose exponent part is returned
598 //
599 // #### Test
600 // ``
601 // ma = library("maths.lib");
602 // ilogb_test = 8.0 : ma.ilogb;
603 // ``
604 //-----
605 ilogb = ffunction(int ilogbf|ilogb|ilogbl (float), <math.h>,"");
606
607
608 //-----`(ma.)log2`-----
609 // Returns the base 2 logarithm of x.
610 //
611 // #### Usage
612 //
613 // ``
614 // _ : log2 : _
615 // ``
616 //
617 // Where:
618 //
619 // * `x`: positive value whose base-2 logarithm is computed
620 //
621 // #### Test
622 // ``
623 // ma = library("maths.lib");
624 // log2_test = 8.0 : ma.log2;
625 // ``
626 //-----
627 log2(x) = log(x)/log(2.0);
628
629
630 //-----`(ma.)expm1`-----
631 // Return exponent of the input signal minus 1 with better precision.
632 //
633 // #### Usage
634 //
635 // ``
636 // _ : expm1 : _
637 // ``
638 //
639 // Where:
640 //
641 // * `x`: input value used for the `exp(x) - 1` computation
642 //
643 // #### Test
644 // ``
645 // ma = library("maths.lib");
646 // expm1_test = 0.5 : ma.expm1;
647 // ``
648 //-----

```

```

649 expm1 = ffunction(float expm1f|expm1|expm1l (float), <math.h>,"");
650
651
652 //-----`(ma.)acosh`-----
653 // Computes the principle value of the inverse hyperbolic cosine
654 // of the input signal.
655 //
656 // #### Usage
657 //
658 // ```
659 // _ : acosh : _
660 // ```
661 //
662 // Where:
663 //
664 // * `x`: input value (greater than or equal to 1)
665 //
666 // #### Test
667 // ```
668 // ma = library("maths.lib");
669 // acosh_test = 1.5 : ma.acosh;
670 // ```
671 //-----
672 acosh = ffunction(float acoshf|acosh|acoshl (float), <math.h>, "");
673
674
675 //-----`(ma.)asinh`-----
676 // Computes the inverse hyperbolic sine of the input signal.
677 //
678 // #### Usage
679 //
680 // ```
681 // _ : asinh : _
682 // ```
683 //
684 // Where:
685 //
686 // * `x`: input value
687 //
688 // #### Test
689 // ```
690 // ma = library("maths.lib");
691 // asinh_test = 0.5 : ma.asinh;
692 // ```
693 //-----
694 asinh = ffunction(float asinhf|asinh|asinh1 (float), <math.h>, "");
695
696
697 //-----`(ma.)atanh`-----
698 // Computes the inverse hyperbolic tangent of the input signal.
699 //
700 // #### Usage
701 //
702 // ```
703 // _ : atanh : _
704 // ```
705 //
706 // Where:
707 //
708 // * `x`: input value in (-1, 1)
709 //
710 // #### Test
711 // ```
712 // ma = library("maths.lib");
713 // atanh_test = 0.5 : ma.atanh;
714 // ```
715 //-----
716 atanh = ffunction(float atanhf|atanh|atanhl (float), <math.h>, "");

```

```

717
718
719 //-----`(ma.)sinh`-----
720 // Computes the hyperbolic sine of the input signal.
721 //
722 // #### Usage
723 //
724 // ```
725 // _ : sinh : _
726 // ```
727 //
728 // Where:
729 //
730 // * `x`: input value
731 //
732 // #### Test
733 //
734 // ma = library("maths.lib");
735 // sinh_test = 0.5 : ma.sinh;
736 //
737 //-----
738 sinh = ffunction(float sinhf|sinh|sinhl (float), <math.h>, "");
739
740
741 //-----`(ma.)cosh`-----
742 // Computes the hyperbolic cosine of the input signal.
743 //
744 // #### Usage
745 //
746 // ```
747 // _ : cosh : _
748 // ```
749 //
750 // Where:
751 //
752 // * `x`: input value
753 //
754 // #### Test
755 //
756 // ma = library("maths.lib");
757 // cosh_test = 0.5 : ma.cosh;
758 //
759 //-----
760 cosh = ffunction(float coshf|cosh|coshl (float), <math.h>, "");
761
762
763 //-----`(ma.)tanh`-----
764 // Computes the hyperbolic tangent of the input signal.
765 //
766 // #### Usage
767 //
768 // ```
769 // _ : tanh : _
770 // ```
771 //
772 // Where:
773 //
774 // * `x`: input value
775 //
776 // #### Test
777 //
778 // ma = library("maths.lib");
779 // tanh_test = 0.5 : ma.tanh;
780 //
781 //-----
782 tanh = ffunction(float tanhf|tanh|tanhl (float), <math.h>,"");
783
784

```

```

785 //-----`(ma.)erf`-----
786 // Computes the error function of the input signal.
787 //
788 // ### Usage
789 //
790 // ```
791 // _ : erf : _
792 // ```
793 //
794 // Where:
795 //
796 // * `x`: input value
797 //
798 // ### Test
799 // ```
800 // ma = library("maths.lib");
801 // erf_test = 0.5 : ma.erf;
802 // ```
803 //-----
804 erf = ffunction(float erff|erf|erfl(float), <math.h>,"");
805
806
807 //-----`(ma.)erfc`-----
808 // Computes the complementary error function of the input signal.
809 //
810 // ### Usage
811 //
812 // ```
813 // _ : erfc : _
814 // ```
815 //
816 // Where:
817 //
818 // * `x`: input value
819 //
820 // ### Test
821 // ```
822 // ma = library("maths.lib");
823 // erfc_test = 0.5 : ma.erfc;
824 // ```
825 //-----
826 erfc = ffunction(float erfcf|erfc|erfcl(float), <math.h>,"");
827
828
829 //-----`(ma.)gamma`-----
830 // Computes the gamma function of the input signal.
831 //
832 // ### Usage
833 //
834 // ```
835 // _ : gamma : _
836 // ```
837 //
838 // Where:
839 //
840 // * `x`: positive input value
841 //
842 // ### Test
843 // ```
844 // ma = library("maths.lib");
845 // gamma_test = 3.0 : ma.gamma;
846 // ```
847 //-----
848 gamma = ffunction(float tgammaf|tgamma|tgamma1(float), <math.h>,"");
849
850
851 //-----`(ma.)lgamma`-----
852 // Calculates the natural logarithm of the absolute value of

```

```

853 // the gamma function of the input signal.
854 //
855 // #### Usage
856 //
857 // ```
858 // _ : lgamma : _
859 // ```
860 //
861 // Where:
862 //
863 // * `x`: positive input value
864 //
865 // #### Test
866 //
867 // ma = library("maths.lib");
868 // lgamma_test = 3.0 : ma.lgamma;
869 // ```
870 //-----
871 lgamma = ffunction(float lgammaf|lgamma|lgammal(float), <math.h>,"");
872
873
874 //-----`(ma.)J0`-----
875 // Computes the Bessel function of the first kind of order 0
876 // of the input signal.
877 //
878 // #### Usage
879 //
880 // ```
881 // _ : J0 : _
882 // ```
883 //
884 // Where:
885 //
886 // * `x`: input value
887 //
888 // #### Test
889 //
890 // ma = library("maths.lib");
891 // J0_test = 1.0 : ma.J0;
892 // ```
893 //-----
894 J0 = ffunction(float j0(float), <math.h>,"");
895
896
897 //-----`(ma.)J1`-----
898 // Computes the Bessel function of the first kind of order 1
899 // of the input signal.
900 //
901 // #### Usage
902 //
903 // ```
904 // _ : J1 : _
905 // ```
906 //
907 // Where:
908 //
909 // * `x`: input value
910 //
911 // #### Test
912 //
913 // ma = library("maths.lib");
914 // J1_test = 1.0 : ma.J1;
915 // ```
916 //-----
917 J1 = ffunction(float j1(float), <math.h>,"");
918
919
920 //-----`(ma.)Jn`-----

```

```

921 // Computes the Bessel function of the first kind of order n
922 // (first input signal) of the second input signal.
923 //
924 // #### Usage
925 //
926 // ```
927 // _ : Jn : _
928 // ```
929 //
930 // Where:
931 //
932 // * `n`: integer order
933 // * `x`: input value
934 //
935 // #### Test
936 // ```
937 // ma = library("maths.lib");
938 // Jn_test = (2, 1.0) : ma.Jn;
939 // ```
940 //-----
941 Jn = ffunction(float jn(int, float), <math.h>,"");
942
943
944 //-----`(ma.)Y0`-----
945 // Computes the linearly independent Bessel function of the second kind
946 // of order 0 of the input signal.
947 //
948 // #### Usage
949 //
950 // ```
951 // _ : Y0 : _
952 // ```
953 //
954 // Where:
955 //
956 // * `x`: positive input value
957 //
958 // #### Test
959 // ```
960 // ma = library("maths.lib");
961 // Y0_test = 1.0 : ma.Y0;
962 // ```
963 //-----
964 Y0 = ffunction(float y0(float), <math.h>,"");
965
966
967 //-----`(ma.)Y1`-----
968 // Computes the linearly independent Bessel function of the second kind
969 // of order 1 of the input signal.
970 //
971 // #### Usage
972 //
973 // ```
974 // _ : Y0 : _
975 // ```
976 //
977 // Where:
978 //
979 // * `x`: positive input value
980 //
981 // #### Test
982 // ```
983 // ma = library("maths.lib");
984 // Y1_test = 1.0 : ma.Y1;
985 // ```
986 //-----
987 Y1 = ffunction(float y1(float), <math.h>,"");
988

```

```

989 //-----`(ma.)Yn`-----
990 // Computes the linearly independent Bessel function of the second kind
991 // of order n (first input signal) of the second input signal.
992 //
993 //
994 // #### Usage
995 //
996 // ```
997 // _ : Yn : _
998 // ```
999 //
1000 // Where:
1001 //
1002 // * `n`: integer order
1003 // * `x`: positive input value
1004 //
1005 // #### Test
1006 //
1007 // ma = library("maths.lib");
1008 // Yn_test = (2, 1.0) : ma.Yn;
1009 //
1010 //-----
1011 Yn = ffunction(float yn(int, float), <math.h>,"");
1012
1013
1014 //-----`(ma.)fabs`, `(ma.)fmax`, `(ma.)fmin`
1015 //-----
1016 // Just for compatibility...
1017 //
1018 // ```
1019 // fabs = abs
1020 // fmax = max
1021 // fmin = min
1022 // ```
1023 //-----
1024 fabs = abs;
1025 fmax = max;
1026 fmin = min;
1027
1028 //-----`(ma.)np2`-----
1029 // Gives the next power of 2 of x.
1030 //
1031 // #### Usage
1032 //
1033 // ```
1034 // np2(n) : _
1035 // ```
1036 //
1037 // Where:
1038 //
1039 // * `n`: an integer
1040 //
1041 // #### Test
1042 //
1043 // ma = library("maths.lib");
1044 // np2_test = 5 : ma.np2;
1045 //
1046 //-----
1047 np2 = -(1) <: >>(1)|_ <: >>(2)|_ <: >>(4)|_ <: >>(8)|_ <: >>(16)|_ : +(1);
1048
1049 //-----`(ma.)frac`-----
1050 // Gives the fractional part of n.
1051 //
1052 // #### Usage
1053 //
1054 // ```
1055 // frac(n) : _

```

```

1056 // ```
1057 //
1058 // Where:
1059 //
1060 // * `n`: a decimal number
1061 //
1062 // #### Test
1063 // ```
1064 // ma = library("maths.lib");
1065 // frac_test = 3.75 : ma.frac;
1066 // ```
1067 //-----
1068 frac(n) = n - floor(n);
1069 decimal = frac;
1070
1071 // NOTE: decimal does the same thing as frac but using floor instead. JOS uses frac a lot
1072 // in filters.lib so we decided to keep that one... decimal is declared though for
1073 // backward compatibility.
1074 // decimal(n) = n - floor(n);
1075
1076
1077 //-----`(ma.)modulo`-----
1078 // Modulus operation using the `(x%y+y)%y` formula to ensures the result is always non-
// negative, even if `x` is negative.
1079 //
1080 // #### Usage
1081 //
1082 // ```
1083 // modulo(x,y) : _
1084 // ```
1085 //
1086 // Where:
1087 //
1088 // * `x`: the numerator
1089 // * `y`: the denominator
1090 //
1091 // #### Test
1092 // ```
1093 // ma = library("maths.lib");
1094 // modulo_test = (-3, 4) : ma.modulo;
1095 // ```
1096 //-----
1097 modulo(x,y) = (x % y + y) % y;
1098
1099
1100 //-----`(ma.)isnan`-----
1101 // Return non-zero if x is a NaN.
1102 //
1103 // #### Usage
1104 //
1105 // ```
1106 // isnan(x)
1107 // _ : isnan : _
1108 // ```
1109 //
1110 // Where:
1111 //
1112 // * `x`: signal to analyse
1113 //
1114 // #### Test
1115 // ```
1116 // ma = library("maths.lib");
1117 // os = library("oscillators.lib");
1118 // isnan_test = (os.osc(1) - 2.0) : sqrt : ma.isnan;
1119 // ```
1120 //-----
1121 isnan = ffunction(int isnanf|isnan|isnanl (float),<math.h>,"");
1122

```



```

1123 //-----`(ma.)isinf`-----
1124 // Return non-zero if x is a positive or negative infinity.
1125 //
1126 // #### Usage
1127 //
1128 // ...
1129 // isinf(x)
1130 // _ : isinf : _
1131 // ...
1132 //
1133 //
1134 // Where:
1135 //
1136 // * `x`: signal to analyse
1137 //
1138 // #### Test
1139 // ...
1140 // ma = library("maths.lib");
1141 // os = library("oscillators.lib");
1142 // isinf_test = (os.impulse - os.impulse) : log : ma.isinf;
1143 // ...
1144 //-----
1145 isinf = ffunction(int isinff|isinfl(float),<math.h>,"");
1146
1147 nextafter = ffunction(float nextafter(float, float),<math.h>,"");
1148
1149 //-----`(ma.)chebychev`-----
1150 // Chebychev transformation of order N.
1151 //
1152 // #### Usage
1153 //
1154 // ...
1155 // _ : chebychev(N) : _
1156 // ...
1157 //
1158 //
1159 // Where:
1160 //
1161 // * `N`: the order of the polynomial, a constant numerical expression
1162 //
1163 // #### Semantics
1164 //
1165 // ...
1166 // T[0](x) = 1,
1167 // T[1](x) = x,
1168 // T[n](x) = 2x*T[n-1](x) - T[n-2](x)
1169 // ...
1170 //
1171 // #### Test
1172 // ...
1173 // ma = library("maths.lib");
1174 // chebychev_test = 0.5 : ma.chebychev(3);
1175 // ...
1176 //
1177 // #### References
1178 //
1179 // * <http://en.wikipedia.org/wiki/Chebyshev\_polynomial>
1180 //-----
1181 chebychev(0,x) = 1;
1182 chebychev(1,x) = x;
1183 chebychev(n,x) = 2*x*chebychev(n-1, x) - chebychev(n-2, x);
1184
1185 //-----`(ma.)chebypoly`-----
1186 // Linear combination of the first Chebyshev polynomials.
1187 //
1188 //
1189 // #### Usage
1190 //

```

```

1191 // ``
1192 // _ : chebychevpoly((c0,c1,...,cn)) : _
1193 // ``
1194 //
1195 // Where:
1196 //
1197 // * `cn`: the different Chebychevs polynomials such that:
1198 // chebychevpoly((c0,c1,...,cn)) = Sum of chebychev(i)*ci
1199 //
1200 // #### Test
1201 // ``
1202 // ma = library("maths.lib");
1203 // chebychevpoly_test = 0.5 : ma.chebychevpoly((1, 0, 1));
1204 // ``
1205 //
1206 // #### References
1207 //
1208 // <http://www.csounds.com/manual/html/chebyshevpoly.html>
1209 //-----
1210 chebychevpoly(lcoef) = _ <: L(0,lcoef) :> _
1211   with {
1212     L(n,(c,cs)) = chebychev(n)*c, L(n+1,cs);
1213     L(n,c)      = chebychev(n)*c;
1214   };
1215
1216
1217 //-----`(ma.)diffn`-----
1218 // Negated first-order difference.
1219 //
1220 // #### Usage
1221 //
1222 // ``
1223 // _ : diffn : _
1224 // ``
1225 //
1226 // Where:
1227 //
1228 // * `x`: input signal
1229 //
1230 // #### Test
1231 // ``
1232 // ma = library("maths.lib");
1233 // os = library("oscillators.lib");
1234 // diffn_test = os.osc(440) : ma.diffn;
1235 // ``
1236 //-----
1237 diffn(x) = x' - x; // negated first-order difference
1238
1239
1240 //-----`(ma.)signum`-----
1241 // The signum function signum(x) is defined as
1242 // -1 for x<0, 0 for x==0, and 1 for x>0.
1243 //
1244 // #### Usage
1245 //
1246 // ``
1247 // _ : signum : _
1248 // ``
1249 //
1250 // Where:
1251 //
1252 // * `x`: input value
1253 //
1254 // #### Test
1255 // ``
1256 // ma = library("maths.lib");
1257 // signum_test = (-5.0) : ma.signum;
1258 // ``

```

```

1259 //-----
1260 signum(x) = (x>0)-(x<0);
1261
1262
1263 //-----`(ma.)nextpow2`-----
1264 // The nextpow2(x) returns the lowest integer m such that
1265 // 2^m >= x.
1266 //
1267 // #### Usage
1268 //
1269 // ```
1270 // 2^nextpow2(n) : _
1271 // ```
1272 // Useful for allocating delay lines, e.g.,
1273 // ```
1274 // delay(2^nextpow2(maxDelayNeeded), currentDelay);
1275 // ```
1276 //
1277 // Where:
1278 //
1279 // * `n`: positive value whose next power-of-two exponent is computed
1280 //
1281 // #### Test
1282 //
1283 // ma = library("maths.lib");
1284 // nextpow2_test = 10.0 : ma.nextpow2;
1285 // ```
1286 //-----
1287 nextpow2(x) = ceil(log(x)/log(2.0));
1288
1289
1290 //-----`(ma.)zc`-----
1291 // Indicator function for zero-crossing: it returns 1 if a zero-crossing
1292 // occurs, 0 otherwise.
1293 //
1294 // #### Usage
1295 //
1296 // ```
1297 // _ : zc : _
1298 // ```
1299 //
1300 // Where:
1301 //
1302 // * `x`: input signal to monitor for zero crossings
1303 //
1304 // #### Test
1305 //
1306 // ma = library("maths.lib");
1307 // os = library("oscillators.lib");
1308 // zc_test = os.osc(440) : ma.zc;
1309 // ```
1310 //-----
1311 zc = ba.tAndH(!=(0)) <: *(mem) < 0;
1312
1313 //-----`(ma.)unwrap`-----
1314 // Unwrap the input signal so that successive output values never differ
1315 // by more than `pi`, switching to a 2*pi-complementary value when needed.
1316 //
1317 // #### Usage
1318 //
1319 // ```
1320 // _ : unwrap(pi) : _
1321 // ```
1322 //
1323 // Where:
1324 //
1325 // * `pi`: maximum discontinuity between the output values (typically `ma.PI`)
1326 //

```

```

1327 // #### Test
1328 // ---
1329 // ma = library("maths.lib");
1330 // os = library("oscillators.lib");
1331 // unwrap_test = os.oscsrc(100) : ma.unwrap(ma.PI);
1332 // ---
1333 //
1334 // #### Example test program
1335 //
1336 // ---
1337 // process = 0 - os.oscsrc(1000) // the true phase is either -PI or +PI
1338 // : an.resonator(1,1000) : !, _ // oscillates between -PI and +PI
1339 // : ma.unwrap(ma.PI); // oscillates near +PI
1340 // ---
1341 //
1342 //-----
1343 unwrap(pi,x) = (_ <: +(dz)) ~ _ with {
1344   dz(z) = wrap(x - z, 2*pi) <: ba.ifNc(1, >(pi), -(2*pi));
1345   wrap(x,d) = x - d * floor(x / d);
1346 };
1347
1348 //-----`ma.)primes`-----
1349 // Return the n-th prime using a waveform primitive. Note that primes(0) is 2,
1350 // primes(1) is 3, and so on. The waveform is length 2048, so the largest
1351 // precomputed prime is primes(2047) which is 17863.
1352 //
1353 // #### Usage
1354 //
1355 // ---
1356 // _ : primes : _
1357 // ---
1358 //
1359 // Where:
1360 //
1361 // * `x`: index of the prime number sequence (0-based).
1362 //
1363 // #### Test
1364 // ---
1365 // ma = library("maths.lib");
1366 // primes_test = 10 : ma.primes;
1367 // ---
1368 //-----
1369 primes(x) = rdtbl(waveform{
1370   2,3,5,7,11,13,17,19,23,29,
1371   31,37,41,43,47,53,59,61,67,71,
1372   73,79,83,89,97,101,103,107,109,113,
1373   127,131,137,139,149,151,157,163,167,173,
1374   179,181,191,193,197,199,211,223,227,229,
1375   233,239,241,251,257,263,269,271,277,281,
1376   283,293,307,311,313,317,331,337,347,349,
1377   353,359,367,373,379,383,389,397,401,409,
1378   419,421,431,433,439,443,449,457,461,463,
1379   467,479,487,491,499,503,509,521,523,541,
1380   547,557,563,569,571,577,587,593,599,601,
1381   607,613,617,619,631,641,643,647,653,659,
1382   661,673,677,683,691,701,709,719,727,733,
1383   739,743,751,757,761,769,773,787,797,809,
1384   811,821,823,827,829,839,853,857,859,863,
1385   877,881,883,887,907,911,919,929,937,941,
1386   947,953,967,971,977,983,991,997,1009,1013,
1387   1019,1021,1031,1033,1039,1049,1051,1061,1063,1069,
1388   1087,1091,1093,1097,1103,1109,1117,1123,1129,1151,
1389   1153,1163,1171,1181,1187,1193,1201,1213,1217,1223,
1390   1229,1231,1237,1249,1259,1277,1279,1283,1289,1291,
1391   1297,1301,1303,1307,1319,1321,1327,1361,1367,1373,
1392   1381,1399,1409,1423,1427,1429,1433,1439,1447,1451,
1393   1453,1459,1471,1481,1483,1487,1489,1493,1499,1511,
1394   1523,1531,1543,1549,1553,1559,1567,1571,1579,1583,

```

1395 1597,1601,1607,1609,1613,1619,1621,1627,1637,1657,
1396 1663,1667,1669,1693,1697,1699,1709,1721,1723,1733,
1397 1741,1747,1753,1759,1777,1783,1787,1789,1801,1811,
1398 1823,1831,1847,1861,1867,1871,1873,1877,1879,1889,
1399 1901,1907,1913,1931,1933,1949,1951,1973,1979,1987,
1400 1993,1997,1999,2003,2011,2017,2027,2029,2039,2053,
1401 2063,2069,2081,2083,2087,2089,2099,2111,2113,2129,
1402 2131,2137,2141,2143,2153,2161,2179,2203,2207,2213,
1403 2221,2237,2239,2243,2251,2267,2269,2273,2281,2287,
1404 2293,2297,2309,2311,2333,2339,2341,2347,2351,2357,
1405 2371,2377,2381,2383,2389,2393,2399,2411,2417,2423,
1406 2437,2441,2447,2459,2467,2473,2477,2503,2521,2531,
1407 2539,2543,2549,2551,2557,2579,2591,2593,2609,2617,
1408 2621,2633,2647,2657,2659,2663,2671,2677,2683,2687,
1409 2689,2693,2699,2707,2711,2713,2719,2729,2731,2741,
1410 2749,2753,2767,2777,2789,2791,2797,2801,2803,2819,
1411 2833,2837,2843,2851,2857,2861,2879,2887,2897,2903,
1412 2909,2917,2927,2939,2953,2957,2963,2969,2971,2999,
1413 3001,3011,3019,3023,3037,3041,3049,3061,3067,3079,
1414 3083,3089,3109,3119,3121,3137,3163,3167,3169,3181,
1415 3187,3191,3203,3209,3217,3221,3229,3251,3253,3257,
1416 3259,3271,3299,3301,3307,3313,3319,3323,3329,3331,
1417 3343,3347,3359,3361,3371,3373,3389,3391,3407,3413,
1418 3433,3449,3457,3461,3463,3467,3469,3491,3499,3511,
1419 3517,3527,3529,3533,3539,3541,3547,3557,3559,3571,
1420 3581,3583,3593,3607,3613,3617,3623,3631,3637,3643,
1421 3659,3671,3673,3677,3691,3697,3701,3709,3719,3727,
1422 3733,3739,3761,3767,3769,3779,3793,3797,3803,3821,
1423 3823,3833,3847,3851,3853,3863,3877,3881,3889,3907,
1424 3911,3917,3919,3923,3929,3931,3943,3947,3967,3989,
1425 4001,4003,4007,4013,4019,4021,4027,4049,4051,4057,
1426 4073,4079,4091,4093,4099,4111,4127,4129,4133,4139,
1427 4153,4157,4159,4177,4201,4211,4217,4219,4229,4231,
1428 4241,4243,4253,4259,4261,4271,4273,4283,4289,4297,
1429 4327,4337,4339,4349,4357,4363,4373,4391,4397,4409,
1430 4421,4423,4441,4447,4451,4457,4463,4481,4483,4493,
1431 4507,4513,4517,4519,4523,4547,4549,4561,4567,4583,
1432 4591,4597,4603,4621,4637,4639,4643,4649,4651,4657,
1433 4663,4673,4679,4691,4703,4721,4723,4729,4733,4751,
1434 4759,4783,4787,4789,4793,4799,4801,4813,4817,4831,
1435 4861,4871,4877,4889,4903,4909,4919,4931,4933,4937,
1436 4943,4951,4957,4967,4969,4973,4987,4993,4999,5003,
1437 5009,5011,5021,5023,5039,5051,5059,5077,5081,5087,
1438 5099,5101,5107,5113,5119,5147,5153,5167,5171,5179,
1439 5189,5197,5209,5227,5231,5233,5237,5261,5273,5279,
1440 5281,5297,5303,5309,5323,5333,5347,5351,5381,5387,
1441 5393,5399,5407,5413,5417,5419,5431,5437,5441,5443,
1442 5449,5471,5477,5479,5483,5501,5503,5507,5519,5521,
1443 5527,5531,5557,5563,5569,5573,5581,5591,5623,5639,
1444 5641,5647,5651,5653,5657,5659,5669,5683,5689,5693,
1445 5701,5711,5717,5737,5741,5743,5749,5779,5783,5791,
1446 5801,5807,5813,5821,5827,5839,5843,5849,5851,5857,
1447 5861,5867,5869,5879,5881,5897,5903,5923,5927,5939,
1448 5953,5981,5987,6007,6011,6029,6037,6043,6047,6053,
1449 6067,6073,6079,6089,6091,6101,6113,6121,6131,6133,
1450 6143,6151,6163,6173,6197,6199,6203,6211,6217,6221,
1451 6229,6247,6257,6263,6269,6271,6277,6287,6299,6301,
1452 6311,6317,6323,6329,6337,6343,6353,6359,6361,6367,
1453 6373,6379,6389,6397,6421,6427,6449,6451,6469,6473,
1454 6481,6491,6521,6529,6547,6551,6553,6563,6569,6571,
1455 6577,6581,6599,6607,6619,6637,6653,6659,6661,6673,
1456 6679,6689,6691,6701,6703,6709,6719,6733,6737,6761,
1457 6763,6779,6781,6791,6793,6803,6823,6827,6829,6833,
1458 6841,6857,6863,6869,6871,6883,6899,6907,6911,6917,
1459 6947,6949,6959,6961,6967,6971,6977,6983,6991,6997,
1460 7001,7013,7019,7027,7039,7043,7057,7069,7079,7103,
1461 7109,7121,7127,7129,7151,7159,7177,7187,7193,7207,
1462 7211,7213,7219,7229,7237,7243,7247,7253,7283,7297,

1463 7307,7309,7321,7331,7333,7349,7351,7369,7393,7411,
1464 7417,7433,7451,7457,7459,7477,7481,7487,7489,7499,
1465 7507,7517,7523,7529,7537,7541,7547,7549,7559,7561,
1466 7573,7577,7583,7589,7591,7603,7607,7621,7639,7643,
1467 7649,7669,7673,7681,7687,7691,7699,7703,7717,7723,
1468 7727,7741,7753,7757,7759,7789,7793,7817,7823,7829,
1469 7841,7853,7867,7873,7877,7879,7883,7901,7907,7919,
1470 7927,7933,7937,7949,7951,7963,7993,8009,8011,8017,
1471 8039,8053,8059,8069,8081,8087,8089,8093,8101,8111,
1472 8117,8123,8147,8161,8167,8171,8179,8191,8209,8219,
1473 8221,8231,8233,8237,8243,8263,8269,8273,8287,8291,
1474 8293,8297,8311,8317,8329,8353,8363,8369,8377,8387,
1475 8389,8419,8423,8429,8431,8443,8447,8461,8467,8501,
1476 8513,8521,8527,8537,8539,8543,8563,8573,8581,8597,
1477 8599,8609,8623,8627,8629,8641,8647,8663,8669,8677,
1478 8681,8689,8693,8699,8707,8713,8719,8731,8737,8741,
1479 8747,8753,8761,8779,8783,8803,8807,8819,8821,8831,
1480 8837,8839,8849,8861,8863,8867,8887,8893,8923,8929,
1481 8933,8941,8951,8963,8969,8971,8999,9001,9007,9011,
1482 9013,9029,9041,9043,9049,9059,9067,9091,9103,9109,
1483 9127,9133,9137,9151,9157,9161,9173,9181,9187,9199,
1484 9203,9209,9221,9227,9239,9241,9257,9277,9281,9283,
1485 9293,9311,9319,9323,9337,9341,9343,9349,9371,9377,
1486 9391,9397,9403,9413,9419,9421,9431,9433,9437,9439,
1487 9461,9463,9467,9473,9479,9491,9497,9511,9521,9533,
1488 9539,9547,9551,9587,9601,9613,9619,9623,9629,9631,
1489 9643,9649,9661,9677,9679,9689,9697,9719,9721,9733,
1490 9739,9743,9749,9767,9769,9781,9787,9791,9803,9811,
1491 9817,9829,9833,9839,9851,9857,9859,9871,9883,9887,
1492 9901,9907,9923,9929,9931,9941,9949,9967,9973,10007,
1493 10009,10037,10039,10061,10067,10069,10079,10091,10093,10099,
1494 10103,10111,10133,10139,10141,10151,10159,10163,10169,10177,
1495 10181,10193,10211,10223,10243,10247,10253,10259,10267,10271,
1496 10273,10289,10301,10303,10313,10321,10331,10333,10337,10343,
1497 10357,10369,10391,10399,10427,10429,10433,10453,10457,10459,
1498 10463,10477,10487,10499,10501,10513,10529,10531,10559,10567,
1499 10589,10597,10601,10607,10613,10627,10631,10639,10651,10657,
1500 10663,10667,10687,10691,10709,10711,10723,10729,10733,10739,
1501 10753,10771,10781,10789,10799,10831,10837,10847,10853,10859,
1502 10861,10867,10883,10889,10891,10903,10909,10937,10939,10949,
1503 10957,10973,10979,10987,10993,11003,11027,11047,11057,11059,
1504 11069,11071,11083,11087,11093,11113,11117,11119,11131,11149,
1505 11159,11161,11171,11173,11177,11197,11213,11239,11243,11251,
1506 11257,11261,11273,11279,11287,11299,11311,11317,11321,11329,
1507 11351,11353,11369,11383,11393,11399,11411,11423,11437,11443,
1508 11447,11467,11471,11483,11489,11491,11497,11503,11519,11527,
1509 11549,11551,11579,11587,11593,11597,11617,11621,11633,11657,
1510 11677,11681,11689,11699,11701,11717,11719,11731,11743,11777,
1511 11779,11783,11789,11801,11807,11813,11821,11827,11831,11833,
1512 11839,11863,11867,11887,11897,11903,11909,11923,11927,11933,
1513 11939,11941,11953,11959,11969,11971,11981,11987,12007,12011,
1514 12037,12041,12043,12049,12071,12073,12097,12101,12107,12109,
1515 12113,12119,12143,12149,12157,12161,12163,12197,12203,12211,
1516 12227,12239,12241,12251,12253,12263,12269,12277,12281,12289,
1517 12301,12323,12329,12343,12347,12373,12377,12379,12391,12401,
1518 12409,12413,12421,12433,12437,12451,12457,12473,12479,12487,
1519 12491,12497,12503,12511,12517,12527,12539,12541,12547,12553,
1520 12569,12577,12583,12589,12601,12611,12613,12619,12637,12641,
1521 12647,12653,12659,12671,12689,12697,12703,12713,12721,12739,
1522 12743,12757,12763,12781,12791,12799,12809,12821,12823,12829,
1523 12841,12853,12889,12893,12899,12907,12911,12917,12919,12923,
1524 12941,12953,12959,12967,12973,12979,12983,13001,13003,13007,
1525 13009,13033,13037,13043,13049,13063,13093,13099,13103,13109,
1526 13121,13127,13147,13151,13159,13163,13171,13177,13183,13187,
1527 13217,13219,13229,13241,13249,13259,13267,13291,13297,13309,
1528 13313,13327,13331,13337,13339,13367,13381,13397,13399,13411,
1529 13417,13421,13441,13451,13457,13463,13469,13477,13487,13499,
1530 13513,13523,13537,13553,13567,13577,13591,13597,13613,13619,

```

1531 | 13627,13633,13649,13669,13679,13681,13687,13691,13693,13697,
1532 | 13709,13711,13721,13723,13729,13751,13757,13759,13763,13781,
1533 | 13789,13799,13807,13829,13831,13841,13859,13873,13877,13879,
1534 | 13883,13901,13903,13907,13913,13921,13931,13933,13963,13967,
1535 | 13997,13999,14009,14011,14029,14033,14051,14057,14071,14081,
1536 | 14083,14087,14107,14143,14149,14153,14159,14173,14177,14197,
1537 | 14207,14221,14243,14249,14251,14281,14293,14303,14321,14323,
1538 | 14327,14341,14347,14369,14387,14389,14401,14407,14411,14419,
1539 | 14423,14431,14437,14447,14449,14461,14479,14489,14503,14519,
1540 | 14533,14537,14543,14549,14551,14557,14561,14563,14591,14593,
1541 | 14621,14627,14629,14633,14639,14653,14657,14669,14683,14699,
1542 | 14713,14717,14723,14731,14737,14741,14747,14753,14759,14767,
1543 | 14771,14779,14783,14797,14813,14821,14827,14831,14843,14851,
1544 | 14867,14869,14879,14887,14891,14897,14923,14929,14939,14947,
1545 | 14951,14957,14969,14983,15013,15017,15031,15053,15061,15073,
1546 | 15077,15083,15091,15101,15107,15121,15131,15137,15139,15149,
1547 | 15161,15173,15187,15193,15199,15217,15227,15233,15241,15259,
1548 | 15263,15269,15271,15277,15287,15289,15299,15307,15313,15319,
1549 | 15329,15331,15349,15359,15361,15373,15377,15383,15391,15401,
1550 | 15413,15427,15439,15443,15451,15461,15467,15473,15493,15497,
1551 | 15511,15527,15541,15551,15559,15569,15581,15583,15601,15607,
1552 | 15619,15629,15641,15643,15647,15649,15661,15667,15671,15679,
1553 | 15683,15727,15731,15733,15737,15739,15749,15761,15767,15773,
1554 | 15787,15791,15797,15803,15809,15817,15823,15859,15877,15881,
1555 | 15887,15889,15901,15907,15913,15919,15923,15937,15959,15971,
1556 | 15973,15991,16001,16007,16033,16057,16061,16063,16067,16069,
1557 | 16073,16087,16091,16097,16103,16111,16127,16139,16141,16183,
1558 | 16187,16189,16193,16217,16223,16229,16231,16249,16253,16267,
1559 | 16273,16301,16319,16333,16339,16349,16361,16363,16369,16381,
1560 | 16411,16417,16421,16427,16433,16447,16451,16453,16477,16481,
1561 | 16487,16493,16519,16529,16547,16553,16561,16567,16573,16603,
1562 | 16607,16619,16631,16633,16649,16651,16657,16661,16673,16691,
1563 | 16693,16699,16703,16729,16741,16747,16759,16763,16787,16811,
1564 | 16823,16829,16831,16843,16871,16879,16883,16889,16901,16903,
1565 | 16921,16927,16931,16937,16943,16963,16979,16981,16987,16993,
1566 | 17011,17021,17027,17029,17033,17041,17047,17053,17077,17093,
1567 | 17099,17107,17117,17123,17137,17159,17167,17183,17189,17191,
1568 | 17203,17207,17209,17231,17239,17257,17291,17293,17299,17317,
1569 | 17321,17327,17333,17341,17351,17359,17377,17383,17387,17389,
1570 | 17393,17401,17417,17419,17431,17443,17449,17467,17471,17477,
1571 | 17483,17489,17491,17497,17509,17519,17539,17551,17569,17573,
1572 | 17579,17581,17597,17599,17609,17623,17627,17657,17659,17669,
1573 | 17681,17683,17707,17713,17729,17737,17747,17749,17761,17783,
1574 | 17789,17791,17807,17827,17837,17839,17851,17863
1575 | }, x);

```